

Prueba de Práctica 1: Conceptos Generales de Programación Avanzada

Programación Avanzada — Evaluación Autocontenida Integrada

Parte 1: Preguntas de Selección Múltiple

Pregunta 1. Eficiencia de estructuras de datos lineales.

Se tiene una aplicación que almacena un flujo constante de tareas pendientes en orden de llegada. Si implementamos esta estructura usando una lista nativa de Python y removemos el primer elemento con `list.pop(0)`, en contraste con una cola doble de la biblioteca estándar usando `collections.deque.popleft()`:

- a) Ambas operaciones tienen una complejidad de tiempo promedio de $\mathcal{O}(1)$.
- b) `list.pop(0)` es $\mathcal{O}(N)$ debido al desplazamiento de elementos en memoria, mientras que `deque.popleft()` es $\mathcal{O}(1)$ gracias a su implementación de lista doblemente enlazada.
- c) `list.pop(0)` es $\mathcal{O}(1)$ y `deque.popleft()` es $\mathcal{O}(N)$ debido al costo de re-enlace de nodos.
- d) Ambas operaciones son $\mathcal{O}(N)$ ya que requieren buscar el elemento en el espacio de direccionamiento.
- e) `list.pop(0)` es $\mathcal{O}(\log N)$ por el costo de reorganización en el árbol interno de memoria de las listas de Python.

Pregunta 2. Excepciones en el Protocolo de Iteración.

Durante la evaluación implícita de un bucle `for` en Python sobre un iterable, ¿cuál es la excepción que levanta el iterador para notificar formalmente que no existen más elementos por recorrer y que el bucle debe terminar de forma limpia?

- a) `IndexError`
- b) `KeyError`
- c) `StopIteration`
- d) `GeneratorExit`
- e) `EOFError`

Pregunta 3. Hilos y Procesos en Python bajo el GIL.

En la implementación de referencia de Python (CPython), el Global Interpreter Lock (GIL) influye de manera determinante en la ejecución paralela de código. ¿Cuál de las siguientes afirmaciones describe de manera correcta el comportamiento del GIL frente a tareas intensivas en CPU y tareas intensivas en I/O (entrada/salida)?

- a) El GIL impide el paralelismo real en tareas de I/O, pero permite paralelismo real en tareas de CPU mediante el uso de hilos nativos.
- b) Tanto las tareas intensivas en CPU como las de I/O se ejecutan en paralelo real en múltiples núcleos del procesador al usar la biblioteca **threading**.
- c) El GIL limita las tareas de I/O a un solo núcleo, pero las tareas de CPU liberan el lock automáticamente permitiendo que múltiples hilos corran simultáneamente.
- d) En tareas intensivas en CPU, el GIL restringe la ejecución a un solo hilo a la vez en un único núcleo físico, mientras que en tareas de I/O (esperas de red, lectura de archivos), los hilos liberan voluntariamente el GIL durante el bloqueo de la llamada de sistema, permitiendo concurrencia efectiva.
- e) El GIL solo se aplica a la biblioteca **multiprocessing**, mientras que la biblioteca **threading** está exenta de esta restricción.

Pregunta 4. El bucle de eventos y el bloqueo de la interfaz gráfica.

En PyQt5, la llamada a `sys.exit(app.exec_())` da inicio formal al ciclo de vida de la interfaz gráfica. ¿Cuál es el rol principal de este bucle de eventos y qué ocurre si ejecutamos una función de cálculo pesado (por ejemplo, procesamiento de matrices o descargas de red secuenciales) directamente en respuesta al clic de un botón sin usar hilos?

- a) El bucle de eventos delega automáticamente las funciones asociadas a botones a núcleos secundarios de la CPU en segundo plano.
- b) El bucle de eventos escucha condicionalmente las entradas de mouse y teclado. Si se ejecuta una tarea pesada en el hilo principal de la GUI, el bucle de eventos no puede procesar los eventos de dibujo o clics entrantes, provocando que la ventana deje de responder ("se congele") ante el sistema operativo.
- c) PyQt5 lanzará una excepción del tipo `GILBlockedException` tras 0.5 segundos de inactividad de dibujo.
- d) La interfaz gráfica se destruirá de inmediato para prevenir la corrupción de memoria del sistema operativo.
- e) La tarea pesada se suspenderá automáticamente para dar prioridad a los eventos gráficos de refresco.

Pregunta 5. Propiedades de la Programación Funcional.

En el paradigma de programación funcional, ¿cuál es la definición formal de una función pura?

- a) Una función que no contiene bucles **for** ni sentencias condicionales **if**.
- b) Una función que no interactúa con variables externas y solo manipula argumentos de tipos numéricos primitivos.

- c) Una función que, dado el mismo conjunto de argumentos, siempre retorna exactamente el mismo resultado y no produce ningún efecto colateral observable.
- d) Una función que está definida anónimamente usando una expresión lambda.
- e) Una función que obligatoriamente implementa herencia de una clase abstracta pura.

Parte 2: Preguntas de Desarrollo

Pregunta 6. Manejo de Excepciones y Clases Personalizadas (Materia: Excepciones e Iteradores).

Implemente una solución que valide lecturas climáticas enviadas desde sensores remotos.

- a) Defina dos excepciones personalizadas heredando de `Exception`: `FormatoSensorInvalidoError` y `LecturaSensorFueraRangoError`.
- b) Escriba una función `parse_sensor_data(linea: str) -> dict` que procese cadenas con el formato `"id_sensor,temperatura,humedad"`. La función debe:
 - Lanzar `FormatoSensorInvalidoError` si la cadena no contiene exactamente tres datos separados por coma o si contiene valores no numéricos.
 - Lanzar `LecturaSensorFueraRangoError` si la temperatura está fuera del rango `[-50.0, 100.0]` o si la humedad está fuera del rango `[0.0, 100.0]`.
 - Retornar un diccionario con los datos limpios y convertidos a tipos apropiados (`str`, `float`, `float`).
- c) Presente un bloque `try-except-else-finally` que simule la llamada a la función procesando datos, manejando individualmente ambas excepciones y asegurando imprimir un mensaje de finalización.

Pregunta 7. Pipelines de Programación Funcional (Materia: Programación Funcional).

Se tiene una lista de transacciones bancarias, donde cada transacción es un diccionario estructurado así: `{"cliente": str, "monto": float, "tipo": str}`. El tipo de transacción puede ser `"compra"`, `"deposito"` o `"giro"`. Escriba un fragmento de código que calcule la suma total de los montos correspondientes exclusivamente a las transacciones de tipo `"compra"`. Requisito Crítico: Debe implementar la lógica utilizando un pipeline de programación funcional que encadene `map`, `filter` y `reduce` junto con expresiones lambda, todo en una única instrucción/línea de código asignada a una variable final. Recuerde importar el módulo necesario para la reducción.

Pregunta 8. Acceso Exclusivo a Recursos con Locks (Materia: Threading y Concurrency).

En una simulación de logística, múltiples camiones (representados por hilos independientes) necesitan descargar mercancía en una única bodega compartida. La bodega tiene una capacidad finita de espacio disponible. Sin embargo, debido a limitaciones físicas, solo un camión puede acceder a la zona de descarga al mismo tiempo para actualizar el inventario.

- a) Diseñe una clase `Bodega` que contenga un inventario inicial de 100 unidades y un objeto `threading.Lock`.
- b) Implemente el método `descargar(self, camion_id, cantidad)` que permita a un camión ingresar su descarga en la bodega, protegiendo la sección crítica con el lock y simulando una demora de 0.1 segundos (usando `time.sleep`) para imitar el tiempo físico de descarga.

- c) Escriba un script que cree y ejecute 5 hilos de camiones diferentes descargando 20 unidades cada uno de forma concurrente, asegurando que el acceso sea exclusivo.

Pregunta 9. Diseño de Ventanas de Inicio de Sesión y Señales (Materia: GUI PyQt5).
Cree una ventana de inicio de sesión utilizando PyQt5. Debe cumplir con:

- a) Una clase `VentanaLogin` que herede de `QWidget`.
- b) Uso de `QFormLayout` para alinear de forma estructurada los campos de entrada de texto: Nombre de Usuario (`QLineEdit`) y Contraseña (`QLineEdit` en modo password).
- c) Un botón de ingreso (`QPushButton`).
- d) Definición de una señal personalizada llamada `login_exitoso` que transporte una cadena de texto (`str`).
- e) Validación: Al hacer clic en el botón, si el usuario es "admin" y la clave es "1234", se debe emitir la señal `login_exitoso` enviando el nombre de usuario. En caso contrario, se debe limpiar la contraseña y cambiar el título de la ventana a "Credenciales Incorrectas".

Solucionario

Respuestas de Selección Múltiple

Pregunta 1: b)

Explicación: Las listas de Python están implementadas internamente como arreglos dinámicos en memoria contigua. Por lo tanto, al eliminar el elemento en el índice 0, todos los elementos siguientes deben desplazarse una posición a la izquierda en memoria, lo cual toma tiempo lineal $\mathcal{O}(N)$. Por otro lado, `collections.deque` está implementado como una lista doblemente enlazada basada en bloques, lo que permite agregar y eliminar elementos en ambos extremos en tiempo constante $\mathcal{O}(1)$ sin desplazar elementos en memoria física.

Pregunta 2: c)

Explicación: El protocolo de iteración en Python especifica que el método especial `__next__()` de un iterador debe lanzar una excepción del tipo `StopIteration` una vez que se agotan los elementos disponibles. El intérprete intercepta esta excepción de forma transparente dentro de la estructura de control del bucle `for` para romper el ciclo sin propagar el error hacia afuera.

Pregunta 3: d)

Explicación: El GIL asegura que solo un hilo de Python ejecute bytecode a la vez en CPython. Para programas con alta carga computacional (CPU-bound), esto significa que los hilos no aprovechan múltiples núcleos en paralelo. Sin embargo, para programas limitados por entrada/salida (I/O-bound), cuando un hilo espera por operaciones del sistema de archivos, sockets de red o temporizadores, CPython libera el GIL de forma segura. Esto permite que otros hilos utilicen el procesador mientras el primero está bloqueado esperando datos, logrando una concurrencia eficiente y útil.

Pregunta 4: b)

Explicación: Las interfaces de usuario de PyQt5 corren en un único hilo (el hilo principal o GUI thread). Este hilo ejecuta un bucle infinito que procesa eventos uno tras otro (redibujar la ventana, mover el mouse, presionar botones). Si un slot conectado a un botón realiza una tarea que toma varios segundos, el hilo de la GUI se queda atrapado ejecutando ese bloque de código y no vuelve al bucle de eventos, impidiendo que la ventana se actualice ante las acciones del usuario y mostrándose congelada.

Pregunta 5: c)

Explicación: Por definición, una función pura en programación funcional cumple dos propiedades: determinismo matemático (el mismo valor de entrada produce siempre el mismo resultado de salida) y ausencia de efectos colaterales (no altera memoria de objetos mutables, variables globales, archivos, consolas, o peticiones de red externas).

Soluciones de Desarrollo

Pregunta 6 (Manejo de Excepciones y Clases Personalizadas):

```
# a) Definición de excepciones personalizadas
class FormatoSensorInvalidoError(Exception):
    """Excepción para cuando la línea no cumple con el formato esperado."""
    pass

class LecturaSensorFueraRangoError(Exception):
    """Excepción para cuando los valores numéricos violan límites físicos."""
    pass

# b) Función validadora y convertidora de datos
def parse_sensor_data(linea: str) -> dict:
    partes = linea.strip().split(',')
    if len(partes) != 3:
        raise FormatoSensorInvalidoError(
            f"Esperado 3 campos, se recibieron {len(partes)}"
        )

    id_sensor = partes[0]
    try:
        temp = float(partes[1])
        hum = float(partes[2])
    except ValueError as err:
        raise FormatoSensorInvalidoError(
            f"Los datos numéricos son inválidos: {err}"
        )

    if not (-50.0 <= temp <= 100.0) or not (0.0 <= hum <= 100.0):
        raise LecturaSensorFueraRangoError(
            f"Límites superados -> temp: {temp}C, hum: {hum}%"
        )

    return {
        "id_sensor": id_sensor,
        "temperatura": temp,
        "humedad": hum
    }

# c) Bloque try-except-else-finally para simulación de lectura
datos_ejemplo = "DHT22,25.5,120.0" # Humedad inválida (> 100%)

try:
    resultado = parse_sensor_data(datos_ejemplo)
```

```

except FormatoSensorInvalidoError as err:
    print(f"[ERROR DE FORMATO]: {err}")
except LecturaSensorFueraRangoError as err:
    print(f"[ERROR DE RANGO]: {err}")
else:
    print(f"[EXITO] Datos parseados: {resultado}")
finally:
    print("[LOG] Intento de procesamiento de sensor finalizado.")

```

Pregunta 7 (Pipelines de Programación Funcional):

```

from functools import reduce

# Datos de prueba
transacciones = [
    {"cliente": "Alice", "monto": 1500.0, "tipo": "compra"},
    {"cliente": "Bob", "monto": 3000.0, "tipo": "deposito"},
    {"cliente": "Charlie", "monto": 500.0, "tipo": "compra"},
    {"cliente": "Alice", "monto": 800.0, "tipo": "giro"}
]

# Pipeline funcional en una sola línea
total_compras = reduce(lambda acc, val: acc + val, map(lambda t: t["monto"], filter(lambda t: t["tipo"] == "compra", transacciones)))

print(f"Total gastado en compras: ${total_compras}") # Imprime: $2000.0

```


Pregunta 8 (Acceso Exclusivo a Recursos con Locks):

```
import threading
import time

class Bodega:
    def __init__(self):
        self.inventario = 100
        # El Lock garantizará acceso exclusivo a la sección crítica
        self.lock = threading.Lock()

    def descargar(self, camion_id, cantidad):
        print(f"[Camión {camion_id}] Esperando turno de descarga...")

        # El bloque 'with' adquiere y libera automáticamente el lock
        with self.lock:
            print(f"[Bodega] >>> Camión {camion_id} INICIA descarga de {cantidad}u.")
            # Sección Crítica
            temp = self.inventario
            time.sleep(0.1) # Simula retraso físico de entrada/salida
            self.inventario = temp + cantidad
            print(f"[Bodega] <<< Camión {camion_id} FINALIZA. Inventario: {self.inventario}u.")

# Creación de la instancia de bodega compartida
bodega_puerto = Bodega()

# Creación de hilos representando a los camiones
hilos = []
for i in range(1, 6):
    hilo = threading.Thread(
        target=bodega_puerto.descargar,
        args=(i, 20)
    )
    hilos.append(hilo)

# Iniciar la ejecución de todos los camiones concurrentemente
for hilo in hilos:
    hilo.start()

# Esperar a que todos los hilos camión terminen
for hilo in hilos:
    hilo.join()

print(f"\n[Puerto] Simulación completada. Inventario final: {bodega_puerto.inventario}u.")
```

Pregunta 9 (Diseño de Ventanas de Inicio de Sesión y Señales):

```
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QFormLayout,
                             QLineEdit, QPushButton)
from PyQt5.QtCore import pyqtSignal

class VentanaLogin(QWidget):
    # Definición de la señal personalizada a nivel de clase
    login_exitoso = pyqtSignal(str)

    def __init__(self):
        super().__init__()
        self.init_ui()

    def init_ui(self):
        self.setWindowTitle("Inicio de Sesión")
        self.resize(300, 150)

        # Crear layout de formulario
        layout = QFormLayout()

        # Configurar campos de texto
        self.input_usuario = QLineEdit()
        self.input_clave = QLineEdit()
        self.input_clave.setEchoMode(QLineEdit.Password) # Ocultar caracteres

        # Agregar filas al formulario
        layout.addRow("Usuario:", self.input_usuario)
        layout.addRow("Contraseña:", self.input_clave)

        # Botón de envío
        self.boton_ingreso = QPushButton("Ingresar")
        self.boton_ingreso.clicked.connect(self.validar_login)
        layout.addRow(self.boton_ingreso)

        self.setLayout(layout)

    def validar_login(self):
        usr = self.input_usuario.text()
        pwd = self.input_clave.text()

        # Lógica de validación
        if usr == "admin" and pwd == "1234":
            self.login_exitoso.emit(usr) # Emisión segura
```

```

        else:
            self.input_clave.clear()
            self.setWindowTitle("Credenciales Incorrectas")

# Demostración del flujo
if __name__ == "__main__":
    app = QApplication(sys.argv)
    ventana = VentanaLogin()

    # Slot para reaccionar al éxito del login
    ventana.login_exitoso.connect(lambda user: print(f"Acceso permitido a: {user}"))

    ventana.show()
    sys.exit(app.exec_())

```